ECE 220 Computer Systems & Programming

Lecture 13 — Problem Solving with Recursion; Recursion with Backtracking

October 8, 2026

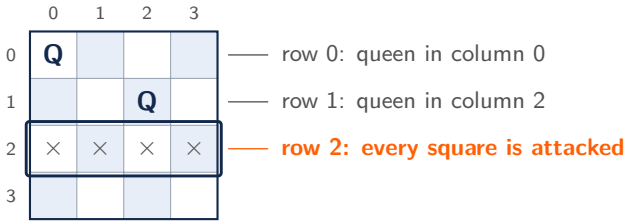
Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

The Dead-End Problem

Place 4 queens on a 4×4 board so that no two share a row, a column, or a diagonal.



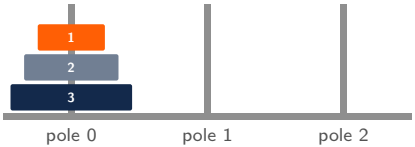
Today's Two Questions

A recursive function needs a smaller problem of the same kind, and a base case.

1. A puzzle states no recurrence. How do we find the smaller problem inside it?
2. A recursive function makes a choice and reaches a dead end. How does it go back and choose again?

Question 1 is Towers of Hanoi. Question 2 is the 4-queens board; the method is recursion with **backtracking**.

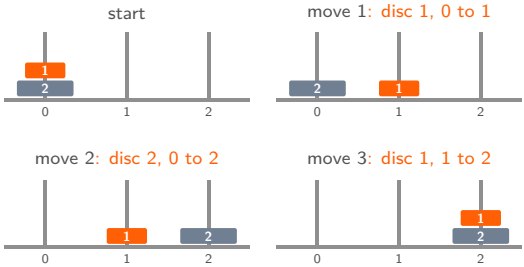
Towers of Hanoi



Move the tower of n discs from pole 0 to pole 2.

- One disc moves at a time: the top disc of one pole goes to the top of another pole
- A disc never rests on a smaller disc

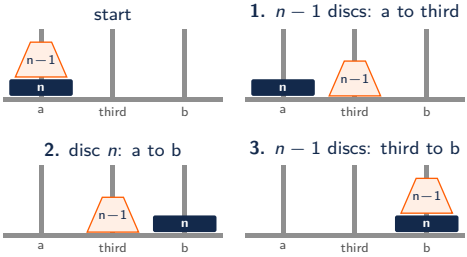
By hand, $n = 2$**.** Draw the poles after each move:



Fewest moves for $n = 1, 2, 3$: 1, 3, 7

In-Class Problem: movetower

`movetower(n, a, b)`: n discs from pole `a` to pole `b`



Steps 1 and 3: this problem again, for $n-1$ discs

The third pole, from a and b: `3 - a - b`

Base case: `n == 1`

```c
/* Move n discs from pole a to pole b.       */
/* a, b are in {0, 1, 2} and a != b, so the */
/* third pole is 3 - a - b:                  */
/*   a = 0, b = 2:  3 - a - b = 1            */
/*   a = 0, b = 1:  3 - a - b = 2            */
/*   a = 2, b = 1:  3 - a - b = 0            */
void movetower(int n, int a, int b) {
    if (n == 1) {                /* base case */
        printf("disc %d: %d -> %d\n", n, a, b);
        return;
    } else {
        movetower(n - 1, a, 3 - a - b);
        printf("disc %d: %d -> %d\n", n, a, b);
        movetower(n - 1, 3 - a - b, b);
    }
}
```

Counting the Moves

`hanoi.c`, run with 3 discs:

```
Number of discs: 3
Move tower of 3 from pole 0 to pole 2
disc 1: 0 -> 2
disc 2: 0 -> 1
disc 1: 2 -> 1
disc 3: 0 -> 2
disc 1: 1 -> 0
disc 2: 1 -> 2
disc 1: 0 -> 2
```

$T(n)$ = number of moves for n discs

n	1	2	3	4	10
$T(n)$	1	3	7	**15**	**1023**

- $T(1) = 1$ and $T(n) = 2\,T(n-1) + 1$
- In closed form, $T(n) = 2^n - 1$
- Most `movetower` records on the run-time stack at one time: n

Every move of movetower is final. In the next problem a choice can turn out wrong, and the function has to take it back.

n-Queens: One Row at a Time

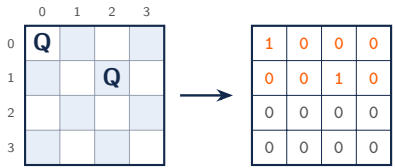
Problem: place N queens on an $N \times N$ board, no two in the same row, column, or diagonal.

1. Start at row 0. Look for a safe column, from left to right, and place a queen there.
2. Found a safe column: make a **recursive call** to place a queen in the next row.
3. Found none: **backtrack**. Return to the previous row, remove its queen, and continue with the next column there.

To backtrack is to undo the most recent choice and try the next alternative.

`int board[N][N];`

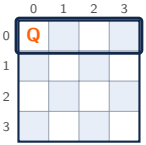
1 = queen, 0 = empty square



Fill in rows 0 and 1 of `board`.

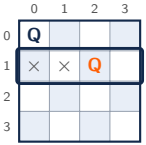
4 Queens by Hand: Queen at (0, 0)

1 row 0



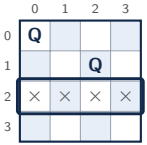
column 0 is safe: queen placed

2 row 1



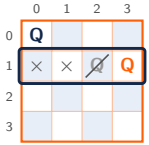
columns 0, 1 attacked; queen in column 2

3 row 2



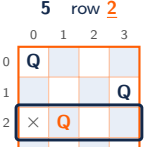
no safe column: `nqueen(board, 2)` returns 0

4 row 1



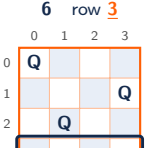
queen removed from column 2; next safe column: 3

5 row 2



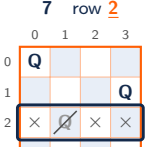
column 0 attacked; queen in column 1

6 row 3



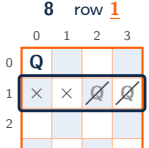
no safe column: returns 0

7 row 2



queen removed; columns 2, 3 attacked: returns 0

8 row 1

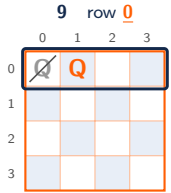


queen removed; no column left: returns 0

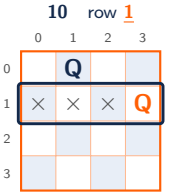
Calls of `nqueen` that have returned 0 so far: 4

× attacked **Q** placed now **Q** placed earlier Q removed

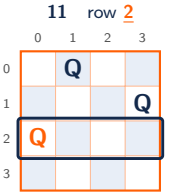
4 Queens by Hand: Queen at (0, 1)



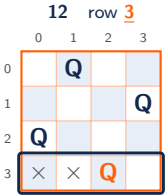
queen removed from column 0; next column: 1



columns 0, 1, 2 attacked; queen in column 3



column 0 is safe: queen placed



columns 0, 1 attacked; queen in column 2

- `nqueen(board, 4)`: no row is left, it returns 1
- Then every waiting call returns 1
- `nqueen` records on the stack at that moment: 5
- Calls of `nqueen` in total: 9

Final `board`:

0	1	0	0
0	0	0	1
1	0	0	0
0	0	1	0

Every row did the same three things: place a queen, call the next row, remove the queen if that call returned 0.

The Backtracking Template

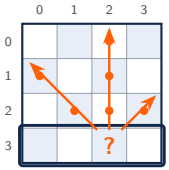
```
int solve(configuration conf) {
    if (no more choices)           /* base case */
        return (conf is a goal state);
    for (all available choices) {
        try one choice c;
        ok = solve(conf with choice c made);
        if (ok)
            return 1;
        else
            unmake choice c;
    }
    return 0;                      /* tried all choices */
}
```

For n-queens:

- configuration: `board` and `row`
- no more choices: `row == N`
- available choices: the safe columns of `row`
- try choice `c`: `board[row][c] = 1`
- unmake choice `c`: `board[row][c] = 0`

Two functions to write: isSafe tests one choice, nqueen is the template.

In-Class Problem: isSafe



Is (3, 2) safe?
Look up the column
and up both diagonals.

- Queens so far are in rows `0 .. row-1`
- Same column: `j == col`
- Diagonal: `i - j == row - col`
- Diagonal: `i + j == row + col`
- Queens at (0,0) and (1,2):
  `isSafe(board, 2, 1)` returns **0**
  $(1 + 2 = 2 + 1)$

```
/* Is board[row][col] safe from the queens  */
/* in rows 0 .. row-1?  1 = safe, 0 = not.  */
int isSafe(int board[N][N], int row, int col) {
    int i, j;
    for (i = 0; i < row; i++)     /* rows above */
        for (j = 0; j < N; j++)
            if (board[i][j] == 1 &&     /* queen */
                (j == col ||            /* col   */
                 i - j == row - col ||  /* diag  */
                 i + j == row + col))   /* diag  */
                return 0;
    return 1;
}
```

In-Class Problem: nqueen

```
/* Place queens in rows row .. N-1.          */
/* 1 = solved, 0 = no placement works.       */
int nqueen(int board[N][N], int row) {
    int c;
    if (row == N)                /* no rows left */
        return 1;
    for (c = 0; c < N; c++) {    /* each column  */
        if (isSafe(board, row, c) == 1) {
            board[row][c] = 1; /* place queen  */
            if (nqueen(board, row + 1) == 1)
                return 1;
            board[row][c] = 0; /* remove it    */
        }
    }
    return 0;                    /* none worked  */
}
```

- `main` calls `nqueen(board, 0)`
- Each call has its own `row` and `c`
- All calls share one `board`, stored in **`main`'s record**

An array is passed as the address of its first element (Lecture 9). A queen placed by one call is on the board for every call below it, and is still there after a call returns. So a call that fails removes its own queen.

Run It: N = 4, Then N = 8

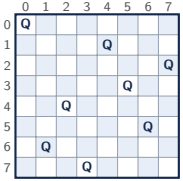
```
int main() {
    int board[N][N], i, j;
    for (i = 0; i < N; i++)     /* empty board */
        for (j = 0; j < N; j++)
            board[i][j] = 0;
    if (nqueen(board, 0) == 1) /* from row 0  */
        printboard(board);
    else
        printf("No solution for N = %d\n", N);
    return 0;
}
```

For 8 queens, change one line: `#define N 8`

$N = 4$ prints

```
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
```

$N = 8$, as a board



- $N = 4$: the board we found by hand
- $N = 8$: 114 calls of `nqueen`; 113 queens placed, 105 removed
- Boards with one queen in each row, $N = 8$: $8^8 = 16{,}777{,}216$

Summary & What's Next

Today

- **Problem solving with recursion**: find the smaller problem of the same kind. `movetower` for n discs is `movetower` for $n-1$, one disc, `movetower` for $n-1$: $2^n - 1$ moves, at most n records on the stack
- **Backtracking**: make a choice, recurse, and undo the choice when the recursive call returns 0
- **Template**: base case; for each choice: make it, recurse, return 1 on success, otherwise unmake it; return 0
- **n-queens**: `isSafe` tests one square against the rows above; `nqueen` fills one row per call; all calls share one `board`

Next lecture

- File I/O in C: streams, `FILE *`, reading and writing files

Code from today: `hanoi.c`, `nqueens.c`, and the skeletons `hanoi_skel.c`, `nqueens_skel.c`.